



Tập 2: CNI Specification

Lý thuyết: Cơ chế ADD/DEL/GC/STATUS và cấu trúc `.conflist`

Network Thực Chiến · Series: Kubernetes Networking · Tập 02

CNI là gì? Tại sao tồn tại?

CNI = Container Network Interface — một **đặc tả kỹ thuật** (specification) mở, không phải sản phẩm cụ thể.

Bài toán: Kubernetes cần giao tiếp với **nhiều hệ thống mạng khác nhau** (Flannel, Calico, Cilium, Weave...) mà không muốn viết code riêng cho từng hệ thống.

```
kubelet — (gọi theo chuẩn CNI) —> CNI Plugin  
          "Làm ơn cấp IP cho Pod này!"    (Flannel / Calico / Cilium...)
```

CNI định nghĩa **giao diện giao tiếp** giữa Container Runtime (kubelet) và Network Plugin.

CNI v1.1.0: Các động từ (Verbs)

CNI spec định nghĩa **4 động từ** mà kubelet có thể gọi vào plugin:

Verb	Khi nào gọi	Mục đích
ADD	Khi tạo Pod	Cấp IP, tạo veth pair, cấu hình routing
DEL	Khi xóa Pod	Giải phóng IP, dọn dẹp veth, xóa routes
CHECK	kubelet sync	Xác minh cấu hình mạng Pod còn đúng không (gọi on-demand, không phải timer độc lập)
GC	Housekeeping	Dọn dẹp tài nguyên mạng của Pod đã mất (mới v1.1.0)
STATUS	Health check	Kiểm tra CNI plugin có sẵn sàng nhận lệnh không (mới v1.1.0)

GC và **STATUS** là 2 verb hoàn toàn mới trong CNI **v1.1.0** — giải quyết vấn đề resource leak khi Node crash.

Luồng ADD khi tạo Pod

```
kubectl apply -f pod.yaml
```

↓
[1] API Server lưu Pod spec vào etcd

↓
[2] Scheduler chọn Node → Ghi nodeSelector vào etcd

↓
[3] kubelet trên Node đó nhận event

↓
[4] kubelet tạo pause container (giữ network namespace)

↓
[5] kubelet đọc /etc/cni/net.d/*.conflist
→ Xác định CNI plugin nào cần gọi

↓
[6] kubelet EXEC binary CNI plugin với verb ADD
Truyền vào: network namespace path, Pod name, container ID

↓
[7] Plugin cấp IP, tạo veth, cấu hình iptables/eBPF
Trả về: JSON chứa IP, routes, DNS info

↓
[8] Pod sẵn sàng với IP! ✅

Cấu trúc File `.conflist` — Chained Plugins

`/etc/cni/net.d/10-flannel.conflist` (ví dụ thực tế):

```
{
  "cniVersion": "1.0.0",
  "name": "cbr0",
  "plugins": [
    {
      "type": "flannel",           ← Plugin chính: cấp IP, tạo veth
      "delegate": {
        "hairpinMode": true,
        "isDefaultGateway": true
      }
    },
    {
      "type": "portmap",          ← Plugin 2: xử lý hostPort mapping
      "capabilities": {
        "portMappings": true
      }
    },
    {
      "type": "bandwidth",        ← Plugin 3: giới hạn bandwidth (optional)
      "capabilities": {
        "bandwidth": true
      }
    }
  ]
}
```

Chained Plugins: Tại sao lại chuỗi?

CNI cho phép **xếp chồng nhiều plugin** — kết quả của plugin trước là đầu vào của plugin sau:

```
kubelet gọi ADD
|
▼
[Plugin 1: bridge]
  ✓ Tạo bridge mylab0
  ✓ Tạo veth pair
  ✓ Cấp IP qua host-local IPAM
  ✓ Thêm routes
    | Kết quả (interface, IP) truyền xuống
    ▼
  [Plugin 2: portmap]
    ✓ Tạo iptables rule cho hostPort mapping
    |
    ▼
  [Plugin 3: firewall]
    ✓ Tạo iptables rule chặn spoofing IP
    |
    ▼
  Trả về JSON kết quả tổng hợp cho kubelet
```

IPAM: Quản lý địa chỉ IP

IPAM (IP Address Management) là sub-plugin chịu trách nhiệm **cấp và thu hồi IP**:

IPAM Plugin	Cơ chế lưu trữ IP đã cấp	Phù hợp với
host-local	File trong <code>/var/lib/cni/networks/</code>	Dev/Lab (đơn giản, per-node)
dhcp	DHCP server truyền thống	Môi trường tích hợp datacenter
calico-ipam	etcd của Calico cluster	Production với Calico CNI
whereabouts	CRD trong K8s	Multus, multi-network

Verb GC: Giải pháp cho Resource Leak

Vấn đề trước CNI v1.1.0: Khi Node bị crash đột ngột:

- Pod bị terminate nhưng CNI không kịp gọi **DEL**
- Kết quả: IP bị giữ mãi trong `/var/lib/cni/networks/`, veth "zombie" còn tồn tại trên Node

Giải pháp — Verb GC (v1.1.0):

```
# kubelet định kỳ gọi GC với danh sách container ID còn sống
# Plugin so sánh với state file, xóa những thứ không còn cần
cnitool gc mylab-network \
  --valid-attachments='[{"containerID":"abc123"}, {"containerID":"def456"}]'
```

GC giống như "garbage collector" — dọn sạch tài nguyên mạng của Pod đã chết mà DEL chưa kịp chạy.

Verb STATUS: Health Check

STATUS (mới từ v1.1.0) cho phép kubelet kiểm tra xem CNI plugin có **sẵn sàng** nhận lệnh không:

```
# Kubelet gọi STATUS trước khi schedule Pod lên Node
cnitool status mylab-network
# Exit code 0 = Plugin OK, sẵn sàng
# Exit code 1 = Plugin đang bị lỗi, KHÔNG schedule Pod lên Node này!
```

Trước đây, nếu CNI plugin bị lỗi (daemon crash, config sai), kubelet vẫn cố tạo Pod và thất bại ở bước ADD — mất thời gian và gây confusion. STATUS giải quyết điều này bằng cách **phát hiện sớm**.

Biến môi trường khi kubelet gọi CNI

Kubelet truyền thông tin cho CNI plugin qua **biến môi trường**:

```
CNI_COMMAND=ADD           # Verb: ADD | DEL | CHECK | GC | STATUS
CNI_CONTAINERID=abc123     # ID của container (pause container)
CNI_NETNS=/proc/123/fd/4   # Path đến network namespace của container
CNI_IFNAME=eth0            # Tên interface sẽ tạo bên trong Pod
CNI_PATH=/opt/cni/bin      # Thư mục chứa CNI binary plugins
CNI_ARGS=K8S_POD_NAME=my-pod;K8S_POD_NAMESPACE=default
```

Plugin nhận config mạng qua **stdin** (JSON format từ file .conflist).

Tổng kết Tập 2

Khái niệm	Tóm tắt
CNI	Đặc tả giao diện giữa kubelet và network plugin
ADD/DEL	Verb cơ bản: cấp IP khi tạo Pod, thu hồi khi xóa
GC	Dọn dẹp resource leak sau Node crash (mới v1.1.0)
STATUS	Health check plugin trước khi schedule Pod (mới v1.1.0)
Chained plugins	Nhiều plugin xếp chồng trong 1 <code>.conflist</code>
IPAM	Sub-plugin quản lý việc cấp/thu hồi IP

Bài Lab 1.2: Tự tay viết `.conflist` và gọi ADD/DEL bằng `cnitool` trực tiếp!

👉 Chuyển sang Lab 1.2

Mở file `lab-guide.md` trong thư mục `1.2/` để thực hành:

- Cài CNI plugin binaries và `cnitool`
- Viết file `.conflist` với chained plugins (bridge + portmap)
- Gọi `cnitool add` thử công và quan sát kết quả
- Gọi `cnitool del` và kiểm tra resource được giải phóng